

Mini Project 1: Packet Analysis Tool

Develop and implement a packet analysis tool to monitor and analyze network traffic for potential security threats.

What Is Packet Analysis : When devices communicate over a network, they send small pieces of data called packets.

1. Open Wireshark – It Is Displaying Available Network Interfaces, Best GUI packet analyzer, It uses Visual packet inspection, Protocol analysis, Deep packet inspection.

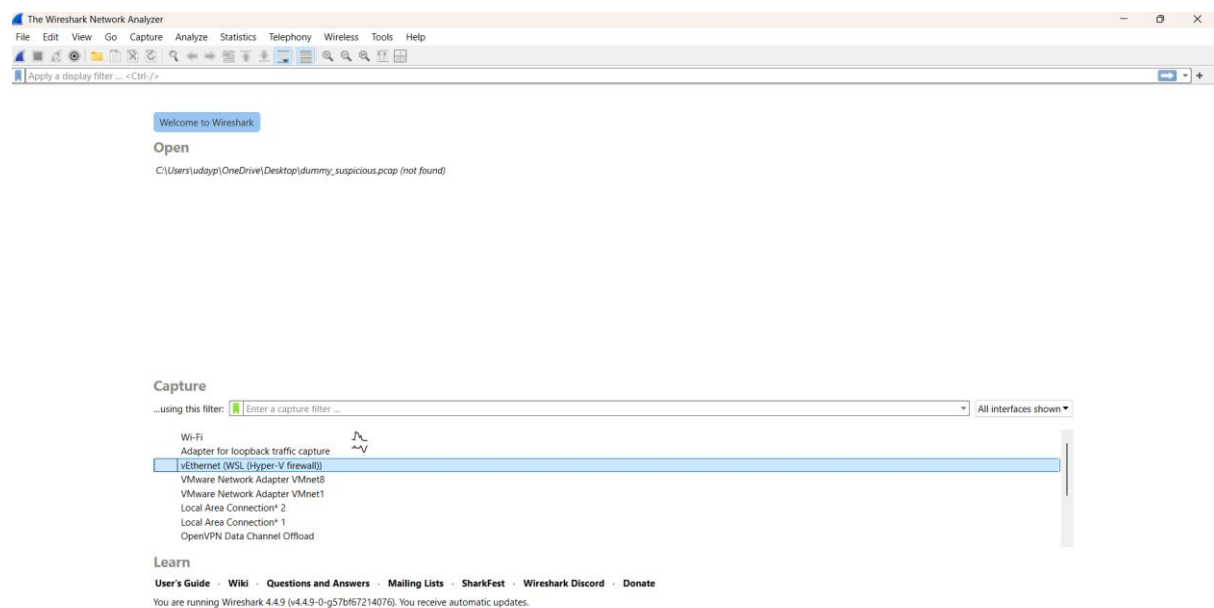


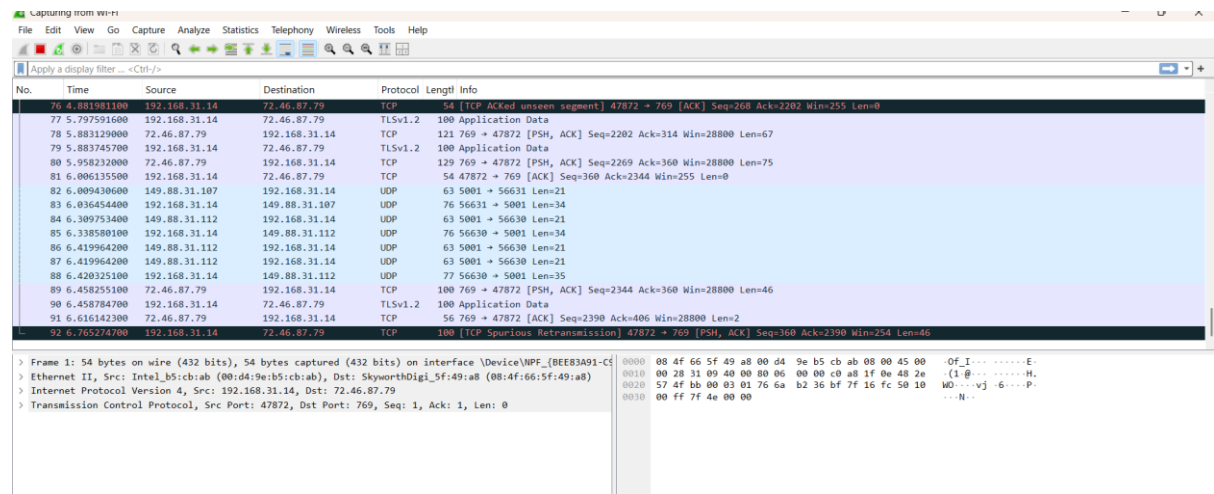
Figure 1 shows the Wireshark home dashboard displaying available network interfaces for packet capturing. Wireshark identifies active interfaces such as Wi-Fi and Ethernet, which are used to monitor network traffic in real time.

2. Now Double click On Any Network Interface wifi or Ethernet



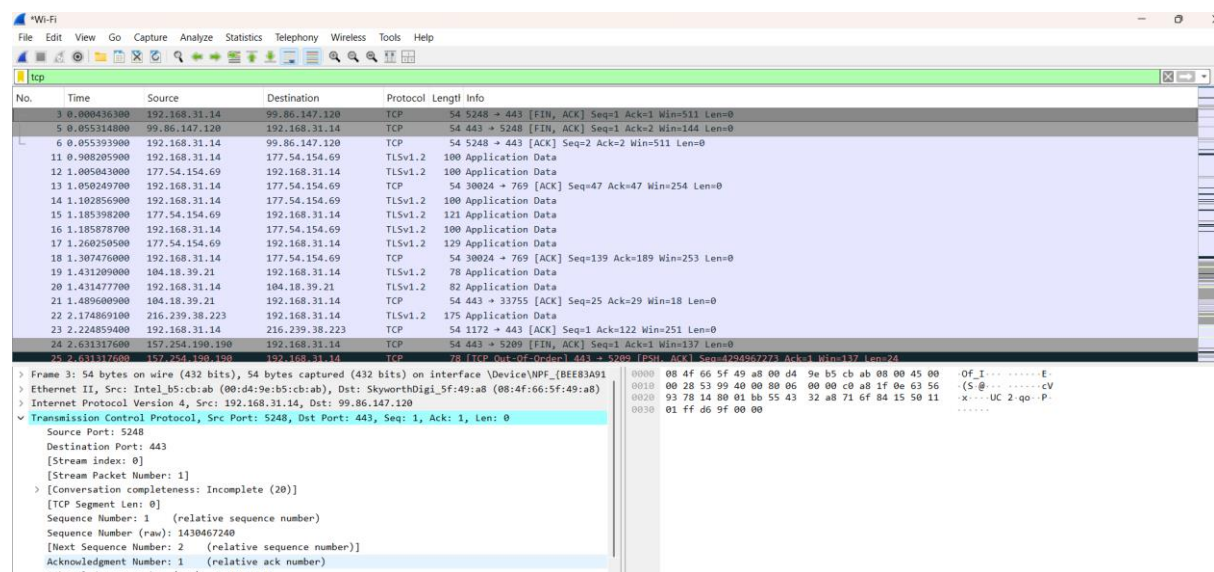
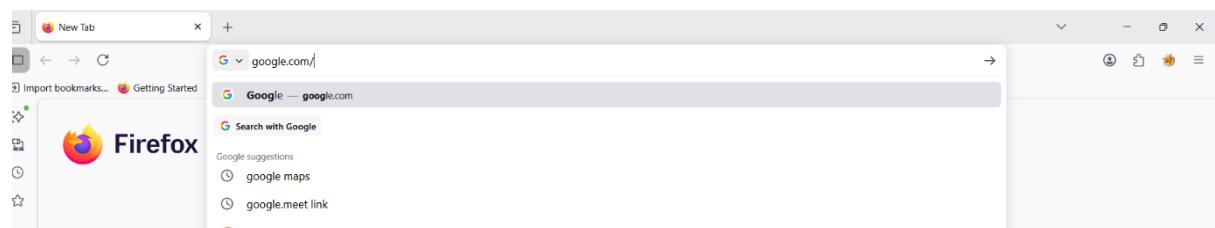
Figure 2 shows the selection of the active Wi-Fi interface for live packet capturing. Once the interface is selected, Wireshark begins monitoring and collecting real-time network packets.

3. The selection of the active Wi-Fi interface for live packet capturing. Once the interface is selected, Wireshark begins monitoring and collecting real-time network packets.



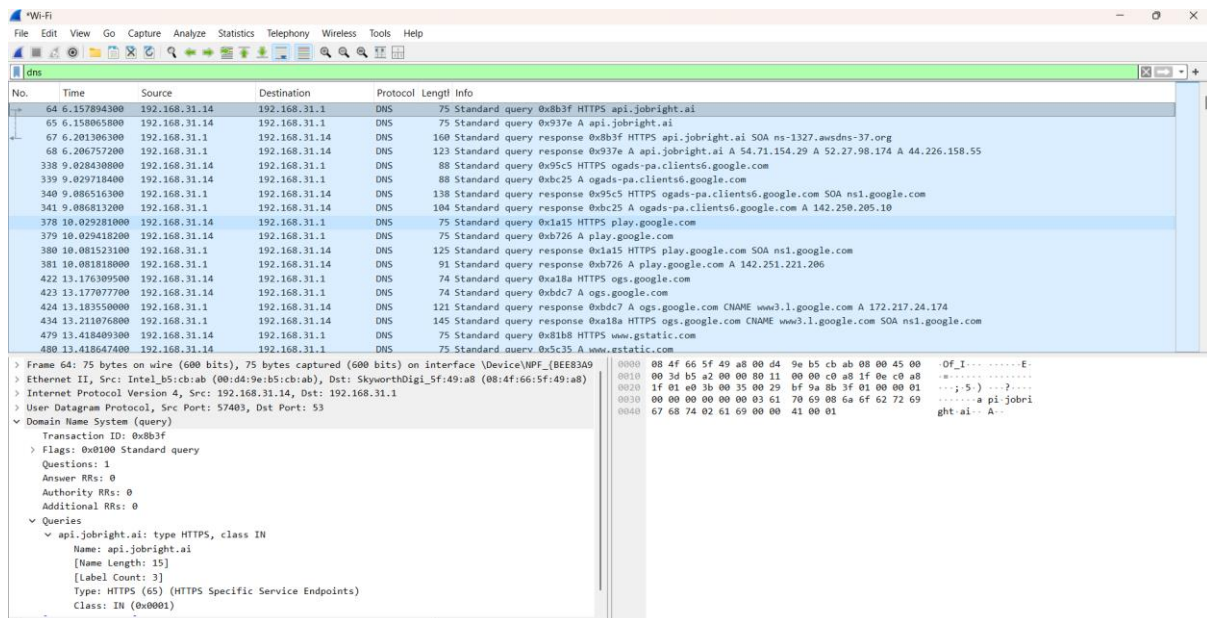
➔ Keep wireshark running for 20-30 Seconds

4. Now open browser visit Google, Open Youtube keep wireshark running This Generates Traffic.



Now you check the Multiple packets flowing , Different Packets Visible Now you enter TCP, DNS, HTTPS

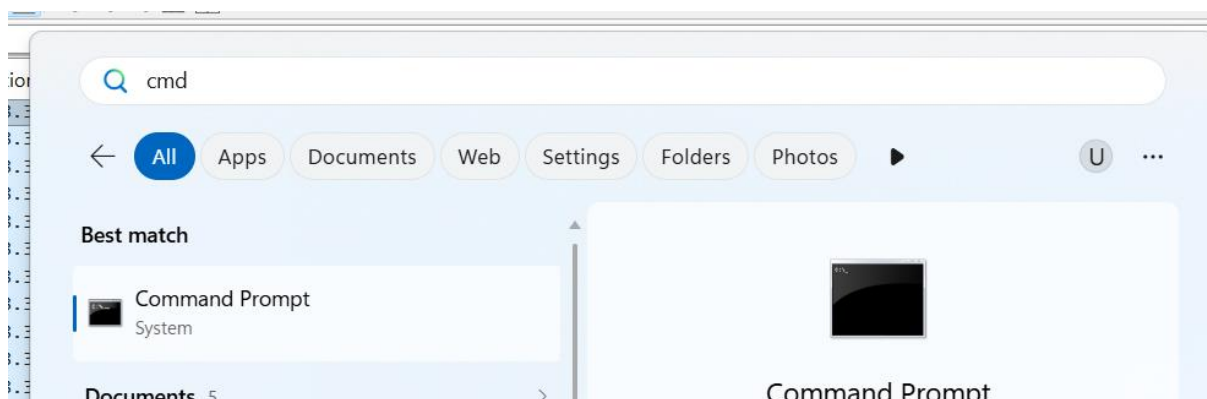
Figure 3 shows live network traffic captured using Wireshark. Multiple protocols such as TCP, DNS, and HTTPS are observed during web browsing activity.



Only DNS packets appear. This DNS Capture DNS Queries, Domain Name Requests.

Figure 4 displays DNS packets filtered using the 'dns' filter in Wireshark. DNS traffic is responsible for translating domain names into IP addresses during internet communication.

5.Open cmd enter “Ping google.com” and wireshark Enter icmp.



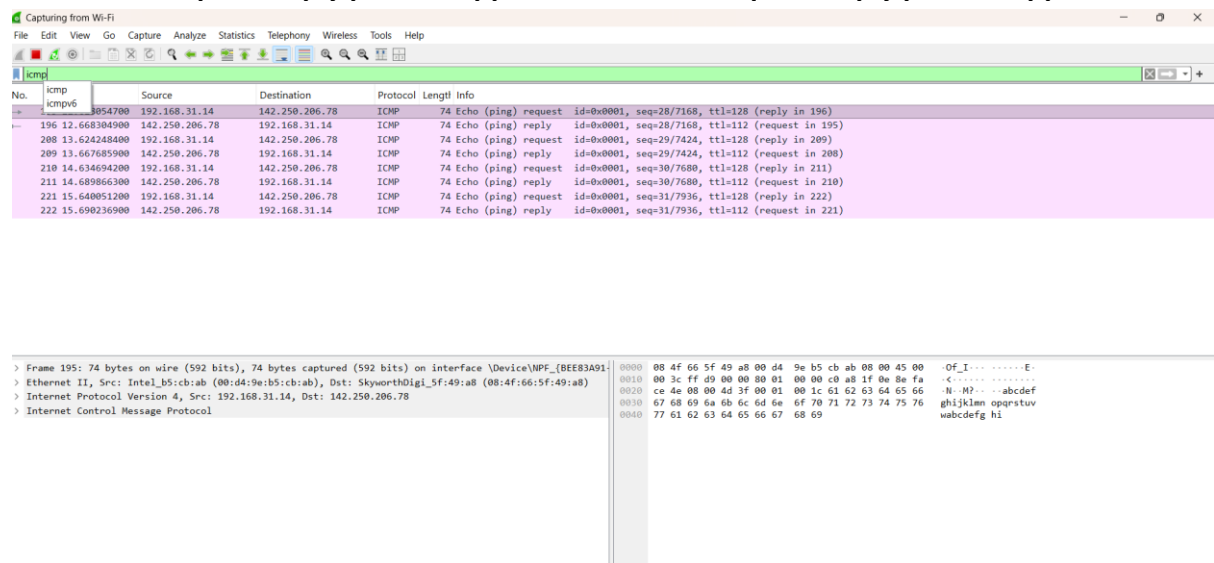
```
Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\udayp>ping google.com

Pinging google.com [142.250.206.78] with 32 bytes of data:
Reply from 142.250.206.78: bytes=32 time=111ms TTL=112
Reply from 142.250.206.78: bytes=32 time=61ms TTL=112
Reply from 142.250.206.78: bytes=32 time=54ms TTL=112
Reply from 142.250.206.78: bytes=32 time=39ms TTL=112

Ping statistics for 142.250.206.78:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 39ms, Maximum = 111ms, Average = 66ms
```

ICMP echo request/reply packets appear. ICMP echo request/reply packets appear.



ICMP echo request/reply packets appear. This Capture Ping terminal ICMP packets in Wireshark

Figure 5 shows ICMP packets generated using the ping command. ICMP is commonly used for connectivity testing and network diagnostics.

6. Now TCP Packet Analysis Enter tcp on wireshark

Wireshark interface showing a list of captured packets. The filter 'tcp' is applied. The packet list shows various TCP segments, including a SYN packet (No. 12473) and a SYN-ACK packet (No. 12474). The packet details pane for the selected packet (No. 12474) shows the TCP header fields: Source Port: 443, Destination Port: 51893, Sequence Number: 1, Acknowledgment Number: 2, and Flags: SYN, ACK.

Capture TCP packets Source/Destination IPs Ports.

Figure 6 shows TCP packet analysis in Wireshark. TCP packets contain source and destination addresses, port numbers, and communication details required for reliable data transmission.

7. Packet Details Pane

Wireshark interface showing a list of captured packets. The filter 'tcp' is applied. The packet list shows various TCP segments, including a SYN packet (No. 12473) and a SYN-ACK packet (No. 12474). The packet details pane for the selected packet (No. 12474) shows the TCP header fields: Source Port: 443, Destination Port: 51893, Sequence Number: 1, Acknowledgment Number: 2, and Flags: SYN, ACK.

Figure 7 displays detailed packet inspection in Wireshark. The packet contains Ethernet, IP, and TCP headers along with payload data used for communication analysis.

8. Suspicious Traffic Detection

Wireshark interface showing a packet capture of a TCP connection. The packet list shows a SYN-ACK packet (Seq=0, Win=65535, Len=0, MSS=1460, WS=256, SACK_PERM). The packet details pane shows the Ethernet II, Internet Protocol Version 4, and Transmission Control Protocol layers. The packet bytes pane shows the raw data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Length	Info
5	0.052994100	192.168.31.14	104.16.249.249	TCP	66	40210 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
6	0.100061300	104.16.249.249	192.168.31.14	TCP	66	443 → 40210 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1282 SACK_PERM WS=8192
22	0.386643100	192.168.31.14	104.16.35.132	TCP	66	40213 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
27	0.456570100	104.16.35.132	192.168.31.14	TCP	66	443 → 40213 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1282 SACK_PERM WS=8192
55	1.219868900	192.168.31.14	49.44.178.91	TCP	66	40216 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
58	1.250431400	49.44.178.91	192.168.31.14	TCP	66	443 → 40216 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1282 SACK_PERM WS=128

Frame 5: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface \Device\NPF_{BEE83A91-C5-00-00-00-00} [eth0] on interface 0.0.0.0

Ethernet II, Src: Intel_b5:cb:ab (00:d4:9e:b5:cb:ab), Dst: Skyworth01gi_5f:49:a8 (08:4f:66:5f:49:a8)

Internet Protocol Version 4, Src: 192.168.31.14, Dst: 104.16.249.249

Transmission Control Protocol, Src Port: 40210, Dst Port: 443, Seq: 0, Len: 0

0000 08 4f 66 5f 49 a8 00 d4 9e b5 cb ab 08 00 45 00 ..Of.I.....E
0010 60 54 22 60 40 00 80 06 00 00 c0 a5 1f 0e 68 104".....h
0020 f9 f9 9d 12 01 b6 43 d1 4b 60 00 00 00 00 80 02C.KI.....
0030 ff ff 41 e7 00 00 02 04 05 b4 01 03 03 08 01 01A.....
0040 ff 02 ..

Figure 8 shows SYN packets filtered using Wireshark. Excessive SYN requests may indicate suspicious activities such as port scanning or reconnaissance attempts.

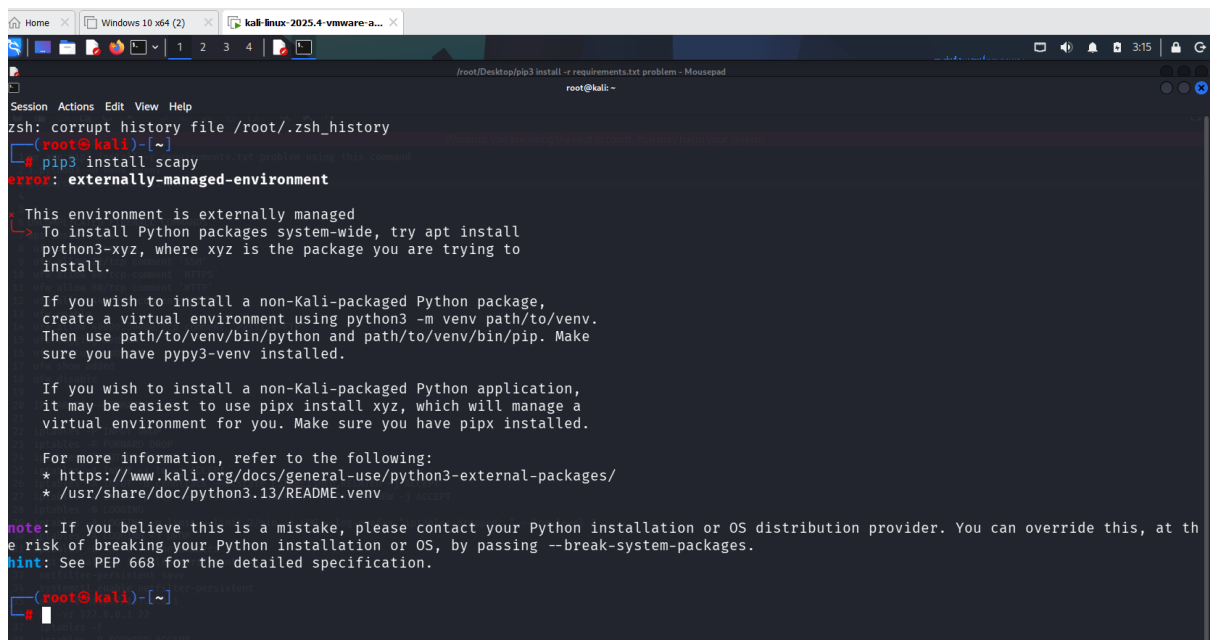
9. Open Terminal/ Linux enter tcpdump -i your network interface like “tcpdump -i eth0”

```
(root@kali)~[~]
# tcpdump -i eth0

tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
03:01:41.958328 IP 192.168.30.131.33125 > ntp1.ggsrv.de.ntp: NTPv4, Client, length 48
03:01:42.036782 IP 192.168.30.131.35760 > 192.168.30.2.domain: 60483+ PTR? 15.192.216.95.in-addr.arpa. (44)
03:01:42.086738 IP 192.168.30.2.domain > 192.168.30.131.35760: 60483 1/0/0 PTR ntp1.ggsrv.de. (71)
03:01:42.088073 IP 192.168.30.131.50985 > 192.168.30.2.domain: 6660+ PTR? 131.30.168.192.in-addr.arpa. (45)
03:01:42.092496 IP 192.168.30.2.domain > 192.168.30.131.50985: 6660 NXDomain 0/0/0 (45)
03:01:42.128220 IP ntp1.ggsrv.de.ntp > 192.168.30.131.33125: NTPv4, Server, length 48
03:01:45.726035 IP 192.168.30.131.49116 > 192.168.30.2.domain: 53855+ PTR? 2.30.168.192.in-addr.arpa. (43)
03:01:45.730461 IP 192.168.30.2.domain > 192.168.30.131.49116: 53855 NXDomain 0/0/0 (43)
03:01:46.417954 IP6 fe80::86cb:5f5d:81ac:c392 > ip6-allrouters: ICMP6, router solicitation, length 8
03:01:46.426810 IP 192.168.30.131.45966 > 192.168.30.2.domain: 20567+ PTR? 2.9.3.c.c.a.1.8.d.5.f.5.b.c.6.8.0.0.0.0.0.0.0.0.0.8.e.f.ip6.arpa. (90)
03:01:46.430662 IP 192.168.30.2.domain > 192.168.30.131.45966: 20567 NXDomain 0/0/0 (90)
03:02:17.799834 IP 192.168.30.131.49446 > ntp1.ggsrv.de.ntp: NTPv4, Client, length 48
03:02:17.980801 IP ntp1.ggsrv.de.ntp > 192.168.30.131.49446: NTPv4, Server, length 48
03:02:26.616868 ARP, Request who-has 192.168.30.2 tell 192.168.30.131, length 28
03:02:26.618373 ARP, Reply 192.168.30.2 is-at 00:50:56:e0:47:6c (oui Unknown), length 46
```

Figure 9 shows live packet capturing using tcpdump in a Linux terminal. Tcpdump is a lightweight command-line packet analyzer commonly used in servers and SOC environments.

10. Scapy Packet Sniffer Output



A terminal window on a Kali Linux system. The user is at the root prompt. They run `pip3 install scapy`, which results in an `error: externally-managed-environment`. The error message explains that the environment is externally managed and suggests using `apt` for system-wide packages or creating a virtual environment with `python3 -m venv` and `pipx` for non-Kali packages. The user is prompted to try `python3-xyz` where `xyz` is the package name. The terminal also shows a note about contacting the Python installation or OS distribution provider and a hint to see PEP 668 for more details.

```
zsh: corrupt history file /root/.zsh_history
(root@kali)~
# pip3 install scapy
error: externally-managed-environment

This environment is externally managed
To install Python packages system-wide, try apt install
python3-xyz, where xyz is the package you are trying to
install.

If you wish to install a non-Kali-packaged Python package,
create a virtual environment using python3 -m venv path/to/venv.
Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
sure you have pypy3-venv installed.

If you wish to install a non-Kali-packaged Python application,
it may be easiest to use pipx install xyz, which will manage a
virtual environment for you. Make sure you have pipx installed.

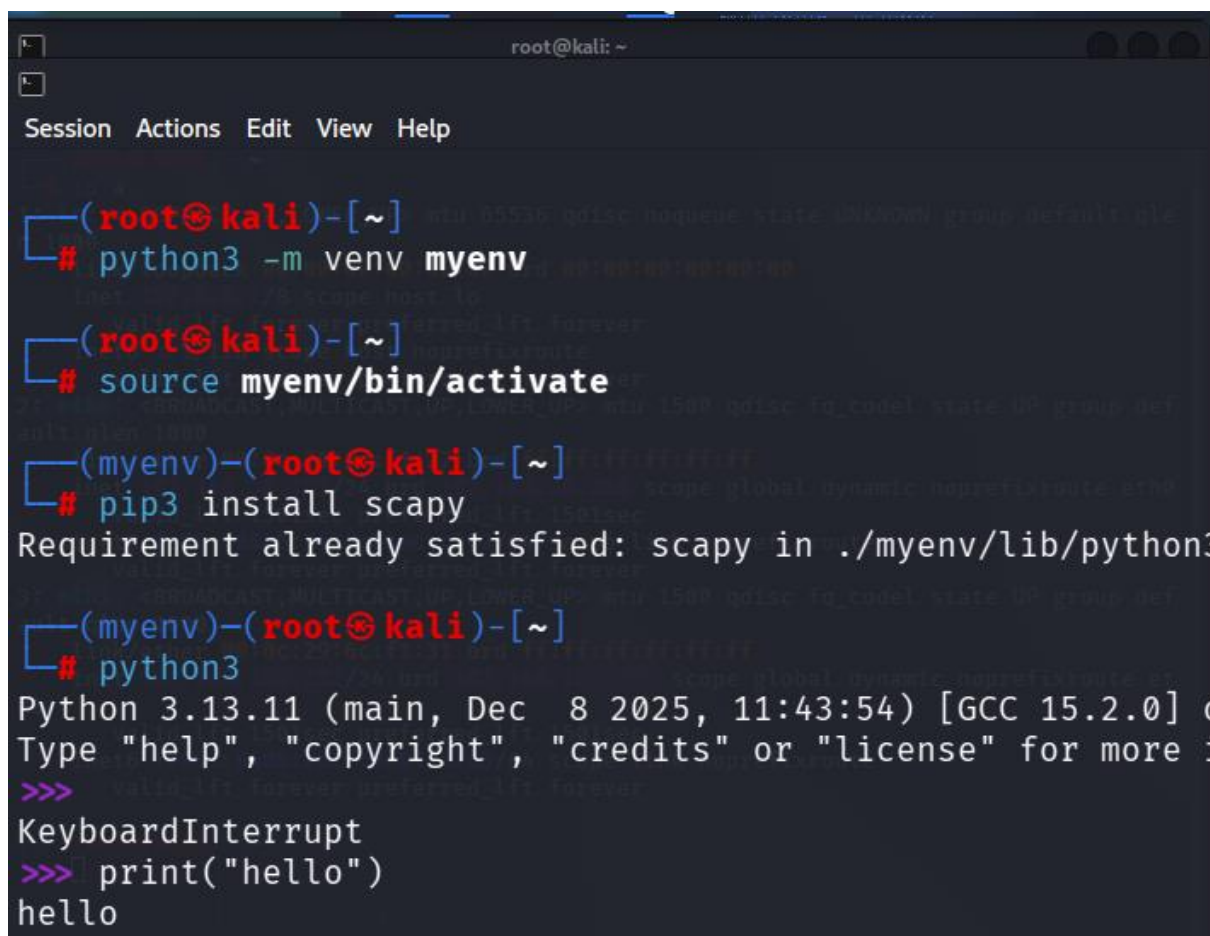
For more information, refer to the following:
* https://www.kali.org/docs/general-use/python3-external-packages/
* /usr/share/doc/python3.13/README.venv

note: If you believe this is a mistake, please contact your Python installation or OS distribution provider. You can override this, at the
risk of breaking your Python installation or OS, by passing --break-system-packages.
hint: See PEP 668 for the detailed specification.

(root@kali)~
#
```

If you get this Error now you can type 2 cmds

1. `python3 -m venv myenv`
2. `source myenv/bin/activate`, and install scapy



A terminal window showing the successful installation and execution of scapy in a virtual environment. The user runs `python3 -m venv myenv` and `source myenv/bin/activate`. Then, they run `pip3 install scapy`, which shows that the requirement is already satisfied. Finally, they run `python3` and enter `print("hello")`, which outputs `hello`.

```
(root@kali)~
# python3 -m venv myenv

(myenv)-(root@kali)~
# source myenv/bin/activate

(myenv)-(root@kali)~
# pip3 install scapy
Requirement already satisfied: scapy in ./myenv/lib/python3.13/site-packages

(myenv)-(root@kali)~
# python3
Python 3.13.11 (main, Dec 8 2025, 11:43:54) [GCC 15.2.0]
Type "help", "copyright", "credits" or "license" for more
>>>
KeyboardInterrupt
>>> print("hello")
hello
```

After install Scapy Then you need to write a code on pyhton3.

```
(myenv)-(root@kali)-[~]
# python3
Python 3.13.11 (main, Dec 8 2025, 11:43:54) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("hello")
hello
>>> sniff(iface="eth0", prn=show, count=10)
Traceback (most recent call last):
  File "<python-input-1>", line 1, in <module>
    sniff(iface="eth0", prn=show, count=10)
NameError: name 'sniff' is not defined
>>> from scapy.all import *
>>> def show(packet):
...     print(packet.summary())
...     sniff(iface="eth0", prn=show, count=10)
...
File "<python-input-3>", line 3
    sniff(iface="eth0", prn=show, count=10)
IndentationError: unexpected indent
>>> from scapy.all import *
...
... def show(packet):
...     print(packet.summary())
...
... sniff(iface="eth0", prn=show, count=10)
...
... 
```

```
Ether / IP / UDP / NTP v4, client
Ether / ARP who has 192.168.30.131 says 192.168.30.2 / Padding
Ether / ARP is at 00:0c:29:6c:f1:27 says 192.168.30.131
Ether / IP / UDP / NTP v4, server
Ether / ARP who has 192.168.30.2 says 192.168.30.131
Ether / ARP is at 00:50:56:e0:47:6c says 192.168.30.2 / Padding

Ether / IP / UDP / NTP v4, client
Ether / IP / UDP / NTP v4, server
Ether / IP / UDP / NTP v4, client
Ether / IP / UDP / NTP v4, server
<Sniffed: TCP:0 UDP:6 ICMP:0 Other:4>
```

Captured packets show normal network communication including ARP requests/replies and NTP (time synchronization) traffic. These packets indicate standard LAN and internet background processes.

Project Overview

In this project, a Packet Analysis Tool was implemented to monitor and analyze live network traffic using tools like Wireshark, tcpdump, and Scapy. The main goal was to understand how data flows in a network and identify potential security-related patterns or anomalies.

◆ What You Did (Implementation Summary)

You successfully performed the following activities:

- Captured live network packets using Wireshark and Scapy
 - Analyzed different types of traffic such as:
 - ARP (Address Resolution Protocol)
 - UDP packets
 - NTP (Network Time Protocol)
 - ICMP (ping traffic)
 - Used filters and interface selection to monitor specific traffic
 - Executed packet sniffing using Python (Scapy automation)
 - Observed real-time packet summaries and network communication behavior
-

◆ Tools Used

- Wireshark (GUI packet analyzer)
 - tcpdump (command-line packet capture)
 - Scapy (Python-based packet sniffing and analysis)
 - Kali Linux environment
-

◆ Key Learnings / Skills Gained

Through this project, you gained practical understanding of:

- Network packet structure (Ethernet, IP, TCP/UDP layers)
- How devices communicate in a local network using ARP
- Real-time traffic monitoring techniques

- Basics of network protocols (DNS, NTP, ICMP, UDP)
 - Packet sniffing automation using Python (Scapy)
 - Troubleshooting issues like interface selection and root access
-

◆ Results / Output Observed

- Successfully captured live network traffic packets
 - Identified normal background communication such as:
 - ARP requests and replies
 - NTP time synchronization packets
 - UDP traffic flows
 - Understood how packets move between devices in a LAN environment
 - Verified real-time packet flow using both GUI (Wireshark) and CLI (Scapy/tcpdump)
-

◆ Final Outcome

The project helped build a strong foundation in network traffic analysis and cybersecurity fundamentals. You were able to successfully capture, inspect, and interpret real network packets, which is a key skill used in SOC (Security Operations Center) and network security roles.

◆ Conclusion (Professional Statement)

This project demonstrates practical hands-on experience in packet analysis, network monitoring, and basic threat observation using industry-standard tools. It builds essential skills required for cybersecurity roles such as SOC Analyst, Network Security Engineer, and Penetration Tester.